

Memoria del Proyecto

Planes Fav

Aplicación web para descubrir y explorar planes de ocio

Autores	Ane Jauregui Saray Guillen Frank Rocha Darío Arenaza
Repositorio	github.com/ane1212/proyecto-planes-fav
Tecnologías	HTML5 · CSS3 · JavaScript ES6+
Vistas	index.html · login.html · details-card.html
Curso	Marzo – 2026

01 Descripción del Proyecto

Idea, objetivo y público destinatario

¿Qué es Planes Fav?

Planes Fav es una aplicación web que permite a los usuarios descubrir, explorar y guardar actividades y planes de ocio de forma sencilla e intuitiva. La plataforma centraliza en un único espacio accesible desde cualquier dispositivo un catálogo de propuestas adaptadas a distintos gustos y preferencias, facilitando la toma de decisiones sobre qué hacer en el tiempo libre.

¿A quién va dirigido?

El proyecto está orientado a cualquier persona que quiera organizar su ocio sin complicaciones: desde quienes buscan actividades para el fin de semana hasta los que desean guardar ideas para una fecha futura. No requiere conocimientos técnicos y está diseñado para ser accesible desde el primer momento.

¿Qué problema resuelve?

La dispersión de información sobre actividades de ocio en múltiples plataformas dificulta la toma de decisiones. Planes Fav centraliza la oferta en una interfaz clara y atractiva, incorporando además datos meteorológicos en tiempo real para que el usuario pueda valorar cuándo y dónde realizar cada actividad.

02 Estructura del Repositorio

Organización de archivos y carpetas del proyecto

El repositorio sigue una arquitectura modular separando vistas, estilos, lógica, componentes y llamadas a API en directorios bien diferenciados. Esto facilita el mantenimiento y la escalabilidad del código.

Estructura del Repositorio

- 📁 **PROYECTO-BUSCAR-PLANES/**
 - 📁 **assets/** — Imágenes y recursos estáticos
 - 📁 **src/** — Código fuente
 - 📁 **diagrams/** — Diagramas de flujos y favoritos
 - 📁 **api/**
 - 📄 apiPlanes.js — Llamadas a la API de planes
 - 📄 apiTiempo.js — Llamadas a la API meteorológica
 - 📁 **components/**
 - 📄 footer.js — Componente de pie de página
 - 📄 nav.js — Componente de navegación
 - 📁 **css/** — Hojas de estilos (index, login, cards)
 - 📁 **js/** — Lógica de aplicación (index, login, storage)
 - 📄 index.html — Página principal
 - 📄 login.html — Acceso de usuarios
 - 📄 details-card.html — Detalle de cada plan
 - 📄 favorites.html — Página favoritos
 - 📄 about.html — Sobre nosotros
 - 📄 README.md — Documentación

Descripción de los directorios

- **assets/** — Almacena todas las imágenes y recursos estáticos utilizados en las vistas.
- **src/api/** — Módulos encargados de la comunicación con servicios externos (API de planes y API meteorológica).
- **src/components/** — Componentes reutilizables de UI: navegación (nav.js) y pie de página (footer.js).
- **src/css/** — Hojas de estilos organizadas por vista: index, login y cards.
- **src/js/** — Lógica de negocio de cada vista y gestión del almacenamiento local.

03 Estructura del Prototipo

Vistas y flujo de navegación entre ellas

Flujo de navegación

La aplicación cuenta con tres vistas principales. El usuario comienza en la pantalla de login, desde donde accede al listado de planes. Al seleccionar un plan concreto se accede a su página de detalle.



Descripción de las vistas

index.html — Listado de planes

Página principal de la aplicación. Muestra el catálogo de planes disponibles en formato de tarjetas (cards) recuperadas desde la API de planes. Incluye la barra de navegación, opciones de filtrado y acceso al detalle de cada actividad. Es el núcleo de la experiencia de usuario.

login.html — Acceso de usuarios

Pantalla de autenticación. Presenta el formulario de acceso con campos de usuario y contraseña. Gestiona la sesión a través de localStorage, redirigiendo al usuario autenticado a la vista principal. Es el punto de entrada obligatorio para acceder a las funcionalidades protegidas de la app.

details-card.html — Detalle del plan

Vista de detalle de un plan concreto. Amplía la información mostrada en la card: descripción completa, imágenes, datos de interés y condiciones meteorológicas en tiempo real obtenidas mediante la API del tiempo. Permite al usuario tomar una decisión informada antes de realizar la actividad.

04 Arquitectura Técnica

Capas del sistema y tecnologías empleadas

Diagrama de capas

La arquitectura del proyecto se organiza en capas claramente diferenciadas, desde la presentación hasta el acceso a datos externos:

Capa	Archivos y responsabilidades
Vistas	index.html · login.html · details-card.html
Estilos	src/css/ → index.css, login.css, cards.css
Lógica	src/js/ → index.js, login.js, storage.js
Compon.	src/components/ → nav.js, footer.js
APIs	src/api/ → apiPlanes.js, apiTiempo.js
Recursos	assets/ → imágenes y recursos estáticos

Tecnologías utilizadas

 HTML5 <i>Estructura semántica</i>	 CSS3 <i>Estilos y layout</i>	 JavaScript <i>ES6+ / Módulos</i>	 APIs <i>Planes + Tiempo</i>
--	---	---	--

Patrones y decisiones técnicas

- Módulos ES6: los archivos .js utilizan import/export para mantener un código limpio y desacoplado.
- Componentes inyectados: nav.js y footer.js insertan el HTML de la navegación y el footer dinámicamente en cada vista.
- Fetch API: las llamadas a apiPlanes.js y apiTiempo.js se realizan con fetch nativo, sin dependencias externas.
- localStorage: la sesión del usuario se almacena en el navegador para persistir entre recargas de página.
- Mobile First: los estilos CSS parten de la resolución móvil y escalan mediante media queries.

05 Enfoque de Desarrollo

Metodología, CSS y buenas prácticas aplicadas

Diseño Mobile First

Todos los estilos se han desarrollado partiendo de la resolución móvil como base, aplicando media queries para adaptar el diseño a pantallas más grandes (tablet y escritorio). Esta estrategia garantiza que la experiencia en dispositivos pequeños sea fluida desde el primer momento.

Selectores CSS

Se han empleado selectores básicos (de clase, elemento e ID) combinados con selectores combinados (descendiente, hijo directo, adyacente) para una especificidad adecuada y un CSS mantenible sin sobreescrituras innecesarias.

Display y Position

Propiedad	Valor	Uso en el proyecto
display	flex	Navegación, tarjetas de planes, botones
display	grid	Layout general de la página de listado
display	none	Ocultar/mostrar elementos dinámicamente
position	sticky	Barra de navegación fija al hacer scroll
position	absolute	Badges y etiquetas sobre las cards
position	relative	Contenedor padre de elementos absolutos

06 Gestión del Trabajo

Distribución de tareas entre los miembros del equipo (Jira)

Actividad	Persona asignada	Estado
Login App - Crear estructura HTML - Crear CSS y responsive - LocalStorage en Login - Crear Registro / Login	Frank Rocha	Finalizado
API – Planes - Investigar cómo implementar API - Implementar / llamar a la API	Ane Jauregui	Finalizado
API – Tiempo - Investigar cómo implementar API - Implementar / llamar a la API - Hacer landing index.html y añadir estilos en css/index.css	Saray Guillen	Finalizado
Página de Favoritos - Añadir la card a favoritos - Usar LocalStorage para visualizar favoritos de usuarios - Diseño del HTML y CSS	Saray Guillen	Finalizado
Detalles de Página - Ejecutar datos de la API y llamada dentro de card de detalles - Creación HTML/CSS de la pagina - Creación card de detalles	Ane Jauregui	Finalizado
Definir Card - Creación y diseño de card - Creación HTML/CSS de la card - Adición datos de API a card	Darío Arenaza	Finalizado
Multifiltros - Integración de APIs y puesta en funcionamiento - Implementación de multifiltro - Comprobación de login y favoritos	Frank Rocha	Finalizado
Documentación - Creación README.md - Creación Memoria técnica	Darío Arenaza	Finalizado
Fixed de la App - Añadir espacio entre la card y el footer - Corrección en posicionamiento de multifiltro - Corrección de errores en contraseña, data e imágenes	Ane Jauregui	Finalizado

07 Conclusiones

Reflexión final y aprendizajes del proyecto

El desarrollo de Planes Fav ha permitido aplicar de forma práctica los conceptos fundamentales del desarrollo web frontend: estructuración semántica con HTML5, maquetación responsive con CSS3 y lógica de interacción con JavaScript.

La organización modular del código — separando vistas, estilos, lógica y componentes — ha resultado clave para mantener un proyecto limpio y escalable. La integración de APIs externas ha añadido una capa de complejidad técnica que ha enriquecido el aprendizaje.

Como próximos pasos, el proyecto podría evolucionar incorporando un sistema de favoritos persistente, filtros avanzados, geolocalización o la posibilidad de valorar los planes.